

Cours 3 — Les fonctions

Semaine 3 ■ Manuel Hachette ch. 2 (§ 2.5) ■ 1 h

Objectifs

- Définir une fonction avec `def`, paramètres et `return`
- Distinguer **définir** et **appeler** ; distinguer `return` et `print`
- Comprendre la notion de variable locale

1. Pourquoi des fonctions ?

Une **fonction** regroupe des instructions sous un nom, pour :

- **réutiliser** un traitement sans recopier le code ;
- **découper** un problème en sous-problèmes ;
- **nommer** les idées du programme (un code lisible se lit comme une recette).

On en utilise depuis la semaine 1 : `print`, `input`, `type`, `int`... Aujourd'hui on écrit les nôtres.

2. Définir et appeler

```
def perimetre(longueur, largeur):
    """Renvoie le périmètre d'un rectangle."""
    p = 2 * (longueur + largeur)
    return p

x = perimetre(5, 3)      # appel : x vaut 16
print(perimetre(10, 2)) # 24
```

- `def nom(paramètres):` puis un bloc indenté — la **définition** n'exécute rien.
- L'**appel** `perimetre(5, 3)` exécute le bloc : `longueur` reçoit 5, `largeur` reçoit 3.
- `return` renvoie le résultat et termine immédiatement la fonction.
- La chaîne sous le `def` est la **docstring** : elle documente la fonction (on approfondira au ch. 8).

3. `return` n'est pas `print`

	<code>return</code>	<code>print</code>
Rôle	renvoie une valeur au programme	affiche à l'écran
Réutilisable ?	oui : <code>2 * perimetre(5, 3)</code>	non, rien n'est renvoyé

```
def double_r(x):
    return 2 * x

def double_p(x):
    print(2 * x)

a = double_r(3)    # a vaut 6
b = double_p(3)    # affiche 6... mais b vaut None !
```

Règle du cours : nos fonctions **renvoient** des valeurs ; l’affichage se fait dans le programme principal.

4. Variables locales

Les paramètres et les variables créées **dans** une fonction sont **locales** : elles n’existent que pendant l’appel, et sont invisibles de l’extérieur.

```
def f(x):  
    y = x + 1  
    return y  
  
f(3)  
print(y)      # NameError : y n'existe pas ici
```

Deux fonctions peuvent donc utiliser le même nom de variable sans conflit.

5. Fonctions à zéro, un ou plusieurs résultats

- Une fonction peut ne rien renvoyer (elle *fait* quelque chose : afficher, dessiner).
- Une fonction booléenne renvoie True/False : `def est_pair(n): return n % 2 == 0`
- `return a, b` renvoie plusieurs valeurs (un tuple — détails au ch. 4).

À retenir

- Définition (`def`) $\neq$ appel : sans appel, rien ne s’exécute.
- `return` renvoie une valeur utilisable ; `print` ne fait qu’afficher.
- Les variables d’une fonction sont locales.
- Une fonction = une tâche, un bon nom, une docstring.